

Homework 2: Lambda Calculus

For your reference, we give the formalization of the λ -calculus (including the evaluation rules) in the appendix at the end of this document.

1. Consider the following term:

$$(\lambda f. (\lambda u. f) (f (\lambda x. x))) ((\lambda x. x)(\lambda x. x))$$

Consider both normal-order evaluation and eager evaluation. For each strategy, give the small-step evaluation sequence. If it terminates, give the complete result, classify it as a canonical form, a normal form, or both, and give a big-step derivation tree. If it diverges, display the repeating small-step.

2. Consider the following term:

$$(\lambda u. \lambda v. (\lambda w. w)v)((\lambda x. x x)(\lambda x. x x))$$

Consider both normal-order evaluation and eager evaluation. For each strategy, give the small-step evaluation sequence. If it terminates, give the complete result, classify it as a canonical form, a normal form, or both, and give a big-step derivation tree. If it diverges, display the repeating small-step.

3. **Nameless representation of terms (de Bruijn indices).**

Bound-variable names are arbitrary: for example, $\lambda x. x$ and $\lambda y. y$ are α -equivalent. De Bruijn indices record which abstraction binds each bound variable occurrence, so α -equivalent closed terms have exactly the same representation. This is useful when implementing the λ -calculus. In this problem, you will see how shifting and substitution avoid variable capture during reduction.

This problem gives another representation of pure λ -terms:

$$(\text{NamelessTerms}) \quad t, s ::= k \mid \lambda. t \mid t s, \quad k \in \{0, 1, 2, \dots\}.$$

An abstraction *encloses* a variable occurrence when the occurrence is inside its body. The *nearest* enclosing abstraction is the innermost such abstraction. Starting there, count outward: indices $0, 1, 2, \dots$ refer to these abstractions in order. These integers are variable references, not Church numerals. Just like the ordinary terms in the appendix, here application

is left associative, and an abstraction's body extends as far to the right as possible.

Translation. A *naming context* Γ assigns indices to variable names. It is written as a finite sequence of variable names, with index 0 at the right. Names may repeat: $\Gamma(x)$ is the number of entries to the right of the rightmost x , and is undefined if x is absent. Write $\text{dom}(\Gamma)$ for the set of names in Γ , and Γ, x for appending x on the right. Thus

$$\begin{aligned}(\Gamma, x)(x) &\stackrel{\text{def}}{=} 0, \\(\Gamma, y)(x) &\stackrel{\text{def}}{=} 1 + \Gamma(x) \quad \text{if } x \neq y.\end{aligned}$$

For each naming context Γ , define the partial translation function

$$\mathcal{D}_\Gamma : \text{Terms} \rightarrow \text{NamelessTerms}, \quad M \mapsto \mathcal{D}_\Gamma(M).$$

The function is defined by

$$\begin{aligned}\mathcal{D}_\Gamma(x) &\stackrel{\text{def}}{=} \Gamma(x), \\ \mathcal{D}_\Gamma(\lambda x. M) &\stackrel{\text{def}}{=} \lambda. \mathcal{D}_{\Gamma, x}(M), \\ \mathcal{D}_\Gamma(M N) &\stackrel{\text{def}}{=} \mathcal{D}_\Gamma(M) \mathcal{D}_\Gamma(N).\end{aligned}$$

The translation $\mathcal{D}_\Gamma(M)$ is defined exactly when $\text{fv}(M) \subseteq \text{dom}(\Gamma)$. The initial context assigns indices to the free variables of M ; for a closed named term, start with $\Gamma = ()$. For example,

$$\mathcal{D}_\emptyset(\lambda x. \lambda y. x y) = \lambda. \lambda. (1 \ 0).$$

When translating the body $x y$, the context is (x, y) , so x has index 1 and y has index 0. For an open-term example,

$$\mathcal{D}_{(x, y)}(\lambda z. x z) = \lambda. (2 \ 0).$$

Here the body is translated in context (x, y, z) : the free variable x has index 2, and the bound variable z has index 0.

Shifting. The operation $\uparrow_c^d t$ shifts indices by d at or above a cutoff c :

$$\begin{aligned}\uparrow_c^d k &\stackrel{\text{def}}{=} \begin{cases} k & k < c, \\ k + d & k \geq c, \end{cases} \\ \uparrow_c^d (\lambda. t) &\stackrel{\text{def}}{=} \lambda. (\uparrow_{c+1}^d t), \\ \uparrow_c^d (t s) &\stackrel{\text{def}}{=} (\uparrow_c^d t) (\uparrow_c^d s).\end{aligned}$$

Here $c \geq 0$ and d is an integer; the operation is defined only when all resulting indices are nonnegative. The cutoff increases under an abstraction to protect the indices bound within the term. For example,

$$\uparrow_0^1 (\lambda. (0 \ 1)) = \lambda. (\uparrow_1^1 (0 \ 1)) = \lambda. (0 \ 2).$$

An index may refer outside a subterm to a surrounding abstraction. The auxiliary computations below may contain such free indices.

Substitution. The operation $t[j := s]$ replaces index j by s :

$$k[j := s] \stackrel{\text{def}}{=} \begin{cases} s & k = j, \\ k & k \neq j, \end{cases}$$

$$(\lambda. t)[j := s] \stackrel{\text{def}}{=} \lambda. (t[j + 1 := \uparrow_0^1 s]),$$

$$(t_1 t_2)[j := s] \stackrel{\text{def}}{=} (t_1[j := s]) (t_2[j := s]).$$

The index to be replaced and the free indices in the replacement both increase when substitution enters an abstraction.

Reduction. Define the reduction relation

$$\rightarrow \subseteq \text{NamelessTerms} \times \text{NamelessTerms}$$

by the following rules:

$$\frac{}{(\lambda. t) s \rightarrow \uparrow_0^{-1} (t[0 := \uparrow_0^1 s])} \quad \frac{t \rightarrow t'}{\lambda. t \rightarrow \lambda. t'}$$

$$\frac{t \rightarrow t'}{t s \rightarrow t' s} \quad \frac{s \rightarrow s'}{t s \rightarrow t s'}$$

The initial shift moves the argument under the binder being removed; the final downward shift accounts for removing that binder.

- (a) Translate each of the following two closed terms into a nameless term.

$$\lambda x. \lambda y. x (\lambda x. y x), \quad (\lambda x. \lambda x. x) (\lambda y. y)$$

- (b) Compute each of the following, showing the intermediate uses of the shifting and substitution rules. Do not perform reduction.

- (i) $\uparrow_0^2 (\lambda. (1 (\lambda. (2 0))))$,
- (ii) $\uparrow_0^{-1} (\lambda. (2 (\lambda. (3 0))))$,
- (iii) $(\lambda. (1 (\lambda. (2 0))))[0 := \lambda. (1 0)]$.

- (c) For each of the following closed nameless terms, first reduce it to a normal form. Show every reduction step and all shifting and substitution calculations. Then give a named term corresponding to the original term, where any choice of fresh bound names is acceptable. Also reduce the named term to a normal form, reducing corresponding applications in the same order. Check that translating the final named term into a nameless term gives the same normal form.

- (i) $\lambda. ((\lambda. \lambda. (1 0)) 0)$,
- (ii) $(\lambda. \lambda. (1 (1 0))) (\lambda. 0)$,
- (iii) $\lambda. ((\lambda. \lambda. (1 (1 0))) (\lambda. (1 0)))$.

- (d) The definition of substitution on nameless terms should agree with the definition of substitution on named terms in the appendix. What theorem needs to be proved to justify this correspondence rigorously? State the theorem formally, using only mathematics (i.e., without texts in natural language). No proof is required.

Appendix

A The λ -Calculus

Syntax:

$$\begin{array}{ll} \text{(Terms)} & M ::= x \mid \lambda x. M \mid M N \\ \text{(Values)} & v ::= \lambda x. M \end{array}$$

Free variables:

$$fv(x) \stackrel{\text{def}}{=} \{x\} \quad fv(M N) \stackrel{\text{def}}{=} fv(M) \cup fv(N) \quad fv(\lambda x. M) \stackrel{\text{def}}{=} fv(M) - \{x\}$$

α -equivalence:

$$\lambda x. M = \lambda y. M[y/x] \quad \text{where } y \text{ fresh}$$

Substitution:

$$\begin{array}{l} x[N/x] \stackrel{\text{def}}{=} N \\ y[N/x] \stackrel{\text{def}}{=} y \\ (M N)[N'/x] \stackrel{\text{def}}{=} (M[N'/x]) (N[N'/x]) \\ (\lambda x. M)[N/x] \stackrel{\text{def}}{=} \lambda x. M \\ (\lambda y. M)[N/x] \stackrel{\text{def}}{=} \lambda y. (M[N/x]), \quad \text{where } y \notin fv(N) \\ (\lambda y. M)[N/x] \stackrel{\text{def}}{=} \lambda z. (M[z/y])[N/x], \quad \text{where } y \in fv(N) \text{ and } z \text{ fresh} \end{array}$$

Reduction rules:

$$\begin{array}{c} \frac{}{(\lambda x. M) N \rightarrow M[N/x]} \qquad \frac{M \rightarrow M'}{\lambda x. M \rightarrow \lambda x. M'} \\[10pt] \frac{M \rightarrow M'}{M N \rightarrow M' N} \qquad \frac{N \rightarrow N'}{M N \rightarrow M N'} \end{array}$$

Zero-or-more steps:

$$\begin{array}{ll} M \rightarrow^0 M' & \text{iff } M = M' \\ M \rightarrow^{k+1} M' & \text{iff } \exists M''. M \rightarrow M'' \wedge M'' \rightarrow^k M' \\ M \rightarrow^* M' & \text{iff } \exists k. M \rightarrow^k M' \end{array}$$

Normal form: a term containing no redex.

Closed term: a term containing no free variables.

Canonical form: a value.

Small-step evaluation.

Normal-order evaluation:

$$\frac{}{(\lambda x. M)N \rightarrow_{\text{NO}} M[N/x]} \quad \frac{M \rightarrow_{\text{NO}} M'}{M N \rightarrow_{\text{NO}} M' N}$$

Eager evaluation:

$$\frac{}{(\lambda x. M)v \rightarrow_{\text{E}} M[v/x]} \quad \frac{M \rightarrow_{\text{E}} M'}{M N \rightarrow_{\text{E}} M' N} \quad \frac{N \rightarrow_{\text{E}} N'}{v N \rightarrow_{\text{E}} v N'}$$

Big-step evaluation.

Normal-order evaluation:

$$\frac{}{v \Rightarrow_{\text{NO}} v} \quad \frac{M \Rightarrow_{\text{NO}} \lambda x. P \quad P[N/x] \Rightarrow_{\text{NO}} v}{M N \Rightarrow_{\text{NO}} v}$$

Eager evaluation:

$$\frac{}{v \Rightarrow_{\text{E}} v} \quad \frac{M \Rightarrow_{\text{E}} \lambda x. P \quad N \Rightarrow_{\text{E}} v_1 \quad P[v_1/x] \Rightarrow_{\text{E}} v_2}{M N \Rightarrow_{\text{E}} v_2}$$